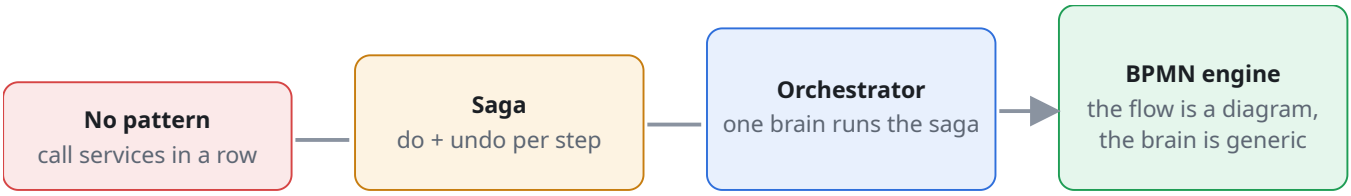
LEARNING NOTES · SAGA-PATTERN-NESTJS

Orchestrators and BPMN engines

The problem each one solves, and what life looks like without them.
Explained with the freight-forwarding demo in this repository.



each step up the ladder takes a class of problems off your hands

1 The business problem: five companies, one shipment

2 Life without any pattern

3 The saga: replacing one transaction with many

4 What the orchestrator solves

5 What the orchestrator still leaves you to build

6 What a BPMN engine solves

7 Side by side, and when to use which

8 Glossary

Code references point at `apps/orchestrator` (hand-coded saga) and `apps/bpmn-engine` (BPMN tool) in this repository.

1 The business problem: five companies, one shipment

A customer asks a freight forwarder to move two containers of solar panels from Shanghai to Rotterdam, then by truck to a warehouse in Duisburg. For the forwarder to say "yes, booked", **five independent parties** all have to agree:

#	PARTY	DO	UNDO
1	Ocean carrier	Book vessel slots CNSHA → NLRTM	Cancel the booking
2	Customs	File the EU import declaration	Withdraw the declaration
3	Trucking	Book trucks Rotterdam → Duisburg	Cancel the truck order
4	Warehouse	Reserve pallet space	Release the space
5	Billing	Charge the customer	Refund

The business rule is simple: **either the whole shipment is booked, or none of it is.** A vessel slot without customs clearance is worthless. A truck booked for containers that will never arrive costs money. Charging a customer for a shipment that can't happen is worse.

In a monolith this is easy

If all five tables lived in one database, you would wrap the work in one transaction. Any failure rolls everything back, and the database guarantees atomicity for you:

```
BEGIN;
  INSERT INTO ocean_bookings …;
  INSERT INTO customs_declarations …;   -- rejected? → ROLLBACK, nothing ever happened
  INSERT INTO truck_bookings …;
  INSERT INTO warehouse_reservations …;
  INSERT INTO invoices …;
COMMIT;
```

In microservices it is impossible

Here each party is a separate service with **its own database**, often owned by another team or another company. There is no shared transaction to roll back. Distributed transactions (two-phase commit) exist on paper, but they lock resources across services while waiting, they need every participant to support the protocol, and external partners won't. In practice, nobody offers them.

> **So the real question is:**
>
> How do you get "all or nothing" across five systems that each commit independently, can each fail, can each be slow, and can each be down? Everything in this document is a different answer to that one question.

2 Life without any pattern

The first version almost everyone writes is a controller that calls the services one after another:

```
async createShipment(order) {
  const ocean     = await oceanCarrier.book(order);
  const customs   = await customs.file(order);         // throws: "HS code needs a licence"
  const trucks    = await trucking.book(order, ocean.eta);
  const warehouse = await warehouse.reserve(order);
  const invoice   = await billing.charge(order, [ocean, customs, trucks, warehouse]);
  return { ocean, customs, trucks, warehouse, invoice };
}
```

It works on the happy path and passes the demo. Then real life arrives:



Figure 1. A failure at step 2 leaves step 1 committed in another company's database. Nobody remembers to cancel it.

What goes wrong

SITUATION	WHAT HAPPENS WITHOUT A PATTERN
A later step fails	Earlier steps stay committed: an orphaned vessel booking, a filed declaration, trucks waiting at the port. You find out from the carrier's invoice.
Someone adds a try/catch	Now every step needs its own cleanup, in reverse order, and the cleanup itself can fail. The code becomes a pyramid of nested catch blocks. Nobody can say if all paths are covered.
A call times out	Did trucking book the trucks or not? A timeout means *unknown*, not *failed*. If you assume "failed" and don't cancel, you may leave a real booking behind.
The server crashes mid-way	The progress lived in local variables, so it is gone. After restart nobody knows the shipment was half booked.
The client retries	The user clicks again, or a load balancer retries. Ocean gets booked twice, and the customer may be charged twice.
Support asks "where is shipment 4711?"	There is no record of which steps ran, so someone has to grep five services' logs.
The process changes	"Customs and trucking can run in parallel", "big orders need a manager's approval", "wait 24 h for the customer to sign": each one is new hand-written control flow in the middle of fragile code.

> **The core issue**
>
> The code describes only the happy path. All the hard parts (partial failure, unknown outcomes, crashes, duplicates, visibility) are left to chance.

3 The saga: replacing one transaction with many

A **saga** accepts that there is no global transaction. It turns the business operation into a **sequence of local transactions** T1 → T2 → … → Tn. Each service commits its own step right away. Every step that can be undone has a **compensating transaction** Ci. If step Tk fails, the saga runs the compensations for the steps that already committed, **in reverse order**.

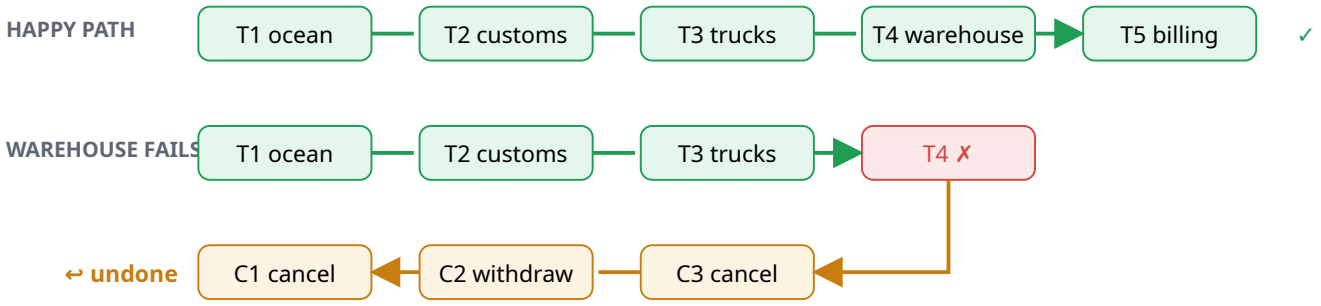


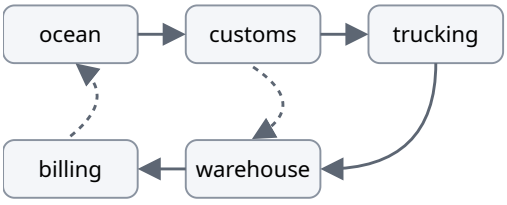
Figure 2. Forward steps commit one by one. On failure, only the steps that committed are compensated, newest first. The failed step T4 committed nothing, so there is nothing to undo.

Three things a saga makes you accept

- **A compensation is not a rollback.** It is a new business action ("cancel the booking"). The booking row stays in the carrier's database with status `CANCELLED`, and the history is visible.
- **The system is only eventually consistent.** For a few hundred milliseconds the vessel slot is booked even though the saga is about to fail. Other users may see that.
- **Order matters.** Cheap, easily undone steps go first. Irreversible or expensive steps go last. Billing is the **pivot**: once the customer is charged nothing after it can fail, so a refund is never needed in the normal flow.

Who runs the saga? Two styles

Choreography: services react to events



eanBooked" → customs listens → "CustomsFiled" → …

Orchestration: one coordinator decides

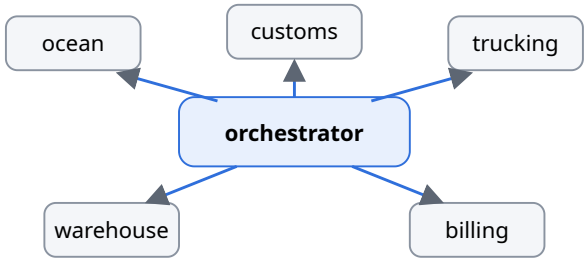


Figure 3. With choreography nobody owns the whole flow; it emerges from event subscriptions. With orchestration one component owns it, and participants only know "do X" and "undo X".

Choreography has no central component, but the flow is scattered across services. Answering "what happens after customs?" or "who cancels the vessel if billing fails?" means reading every service. This repository implements **orchestration**.

4 What the orchestrator solves

The orchestrator (`apps/orchestrator`) is the forwarder's central brain. It owns the saga definition, sends each participant a command over TCP, and above all **writes down what happened after every step**. It turns "call five services and hope" into a **persisted, recoverable, observable** sequence with a defined way to undo it. Each failure mode from section 2 gets an explicit answer:

PROBLEM FROM SECTION 2	HOW THE ORCHESTRATOR HANDLES IT	WHERE
A later step fails	Runs C_{k-1} … C1 in reverse order, using the list of completed steps it saved	`saga-orchestrator.service.ts`
Messy nested try/catch	The flow is **data**: an ordered list of (Ti, Ci) pairs. A single generic loop runs it forwards and backwards	`shipment-saga.definition.ts`
Timeout: unknown outcome	Treats a timeout as "might have happened" and **also compensates the step that timed out**. A clear rejection or `ECONNREFUSED` means nothing happened, so that step is skipped	`participant-clients.ts` , `lastStepOutcomeUnknown`
Server crash mid-way	The saga row and step log are persisted after every step. On boot, unfinished sagas are found and recovered by undoing everything that might have happened (backward recovery)	`saga-recovery.service.ts`
Duplicate requests and retries	Every participant keys its row by `sagaId` . A repeated "book" returns the existing booking, and a repeated "cancel" returns `ALREADY_CANCELLED`	each participant's `*.service.ts`
A cancel overtakes its book	The participant stores a `CANCELLED` "tombstone", so a late "book" for that saga is refused instead of creating an orphan	`cancel()` in participants
Compensation itself fails	Retries with backoff. If it still fails the saga becomes `COMPENSATION_FAILED` , and a human calls `POST /shipments/:id/retry-compensation`	`compensate()` , `retryCompensation()`
"Where is shipment 4711?"	`GET /shipments/:id` shows status, timeline and context. A live UI streams every command and reply	`shipments.controller.ts` , `public/index.html`

The pieces that make it work

A saga log

A `sagas` row with `status` and `completedSteps` , plus one `saga_step_logs` row per attempt. The database, not memory, is the source of truth.

A state machine

`STARTED → COMPLETED` , or `STARTED → COMPENSATING → COMPENSATED` , or `→ COMPENSATION_FAILED` (manual retry).

Context passing

Results of earlier steps feed later ones: trucking plans the pick-up from the vessel ETA, billing builds the invoice from every step's cost.

Dumb participants

Services know nothing about the saga or each other. They only implement an idempotent "do" and "undo".

5 What the orchestrator still leaves you to build

The hand-coded orchestrator is a good fit for *this* saga: five steps in a fixed order that finish within seconds. Look at what happens when the business asks for more. Every request below is reasonable, and every one means writing more engine code:

THE BUSINESS ASKS…	WHAT YOU'D HAVE TO ADD TO THE ORCHESTRATOR
"Run customs and trucking in parallel."	Fork/join logic, tracking several in-flight steps, and compensating a branch that is still running when the other one fails.
"Orders over €10,000 need a manager's approval."	Conditional branching in the definition, and an expression language for the rule.
"Wait until the manager clicks Approve, possibly tomorrow."	A **wait state**: persist the saga mid-flight, a task inbox, an API to resume it, and no thread blocked for a day.
"If the customer hasn't signed in 48 hours, escalate."	Durable timers that survive restarts, and a scheduler that polls for due timers.
"Change the order of steps, but don't break the 300 shipments in flight."	Versioned definitions: running sagas must finish on the old flow while new ones use the new flow.
"Ops needs to fix a stuck shipment without calling a developer."	An operations UI to inspect, edit data, retry or cancel *any* process, not only shipments.
"Now build the same for returns, claims and customs audits."	Copy the engine, or generalise it into… a workflow engine.
"Show the business analyst what the process does."	Drawing and maintaining a diagram by hand that drifts away from the code.

> **The pattern behind all of these**
>
> Each request adds generic process machinery (branching, waiting, timers, versioning, operations) to code that was meant to be about shipping. Do it a few times and you end up writing a workflow engine by accident, usually a worse one, with the business rules tangled inside it.

That is exactly the gap BPMN engines fill. They separate two things the orchestrator mixes together:

- **The process**, meaning what happens, in which order, under which conditions, and who waits for what. This becomes a **diagram**.
- **The work**, meaning how to book a vessel or charge a card. This stays **code** (small handlers).

The machinery that runs any diagram (tokens, persistence, timers, retries, versioning, operations UI) is written **once**, in the engine, and reused for every process.

6 What a BPMN engine solves

BPMN 2.0 is an OMG standard that is both a notation people can read (circles for events, boxes for tasks, diamonds for decisions) and an XML format with exact execution semantics. A BPMN tool has a **modeler** (draw), an **engine** (run) and a **cockpit** (operate). `apps/bpmn-engine` has all three.

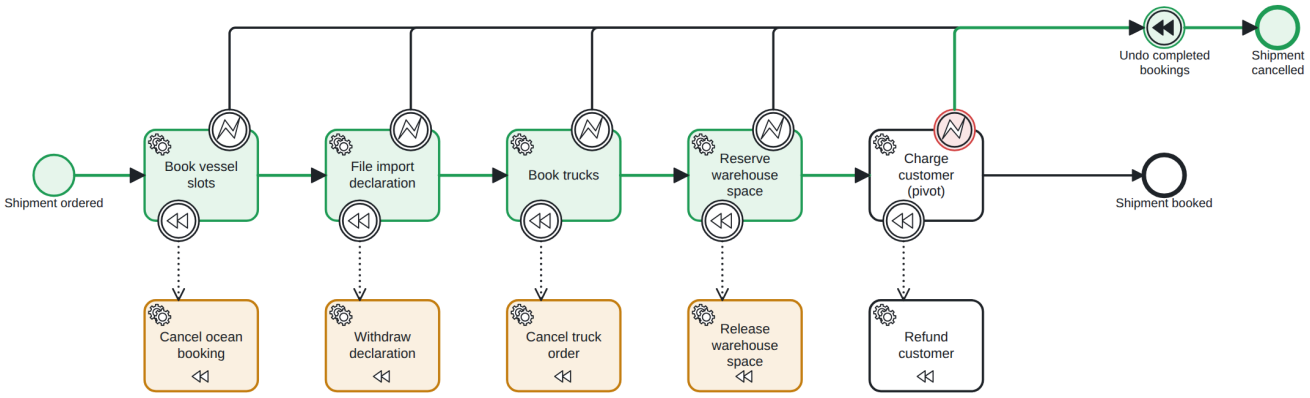


Figure 4. The shipment saga as a BPMN diagram, in this repo's cockpit after the "credit limit" scenario. Green: completed. Red: billing rejected the charge (error boundary event). Orange: compensations the engine ran, in reverse order. Refund didn't run, because billing never completed.

How the earlier problems map onto BPMN

NEED	BPMN ANSWER	IN THE ENGINE
Undo on failure (the saga)	Error boundary event ⚡ + compensation event ⏪ linked to an undo task	`compensate()` walks completed activities backwards
Branching	Exclusive gateway ◇ with conditions and a default flow	`chooseExclusiveFlow()`
Parallel steps	Parallel gateway ◇+ (fork and join)	`executeParallelGateway()`
Human approval	User task, a wait state with a form	token `WAITING` until `completeUserTask()`
"Wait 48 hours"	Timer event	persisted `dueAt`, `fireDueTimers()`
Crash safety	Engine persists every instance after every step	`save()`, `resumeAfterRestart()`
Service down or timing out	**Incident**: the token stops and an operator retries	`incident()`, `retryIncident()`
Changing the process safely	Deploy a new version; running instances keep theirs	`deploy()`, `definition(key, version)`

The core idea: token passing

An instance holds **tokens**, each sitting on a node. The engine picks a ready token and executes its node (run a handler, evaluate a gateway, fork, or **park** at a user task or timer), then saves. When every token is parked or gone it stops. Nothing lives in memory, so an instance can wait a week.

> **Business errors vs. technical errors**
>
> `throw new BpmnError("BOOKING_REJECTED")` is an outcome the *diagram* handles ("vessel full" → compensation path). Any other exception (service down, bug, timeout) becomes an **incident** for a human. The hand-coded orchestrator mixes the two in one `catch`.

7 Side by side, and when to use which

	NO PATTERN	HAND-CODED ORCHESTRATOR	BPMN ENGINE
Partial failure	**orphaned bookings**	**reverse compensation**	**compensation events**
Unknown outcome (timeout)	**ignored**	**compensates it too**	**incident, operator decides**
Crash mid-way	**progress lost**	**saga log + recovery**	**instance persisted per step**
Duplicates	**double booking**	**idempotent participants**	**same (at-least-once handlers)**
Visibility	**grep logs**	**custom UI per saga**	**generic cockpit**
Branching / parallel	**hand-written**	**extend the engine**	**gateways**
Human steps, timers	**very hard**	**build wait states**	**user tasks, timer events**
Changing the flow	**edit code**	**edit code, careful with in-flight sagas**	**redraw + new version**
Readable by non-developers	**no**	**no**	**the diagram is the spec**
Extra concepts to learn	**none**	**few**	**BPMN, engine, operations**
Infrastructure	**none**	**your own service**	**an engine to run and upgrade**
Testing the logic	**plain unit tests**	**plain unit tests**	**split between diagram and handlers**

A rule of thumb

A hand-coded orchestrator is enough when…

- the flow is short and mostly linear,
- it finishes in seconds or minutes,
- only developers need to understand it,
- you have one or two such processes.

A BPMN engine pays off when…

- processes run for hours, days or weeks,
- people take part (approvals, reviews),
- there are timers, deadlines, escalations,
- the flow changes often, or must be explained to the business or auditors,
- you have many processes and want one way to run and operate them all.

Never acceptable: no pattern at all

If an operation touches more than one service that commits independently, you need at least a saga with persisted state and idempotent participants. Without it, partial failures become silent data corruption.

Where to go next in this repo

1. Run `npm run start:all` (orchestrator, :3000) and `npm run start:bpmn` (BPMN tool, :3200). Trigger the same failure in both and compare. More exercises are in `apps/bpmn-engine/README.md`.
2. In the BPMN modeler, move "Book trucks" after the warehouse and deploy v2. Doing the same in the orchestrator means a code change.
3. Add a user task "Ops double-checks the booking" before billing. Think about what the hand-coded orchestrator would need for that.

8 Glossary

TERM	MEANING
Local transaction	A commit inside one service's own database. The only kind of atomicity you get in microservices.
Saga	A business operation made of local transactions, each with a compensating action, so the whole can be undone semantically.
Compensation	A new business action that undoes the *effect* of a committed step (cancel, refund, release). Not a database rollback.
Pivot transaction	The point of no return. Put irreversible or costly steps (charging money) last, so nothing after them can fail.
Choreography	Saga style where services react to each other's events. No central owner of the flow.
Orchestration	Saga style where one coordinator sends commands and owns the flow and its state.
Saga log	The orchestrator's persisted record of which steps ran. It tells you what to undo after a failure or crash.
Idempotency	Doing an operation twice has the same effect as doing it once. This is what makes retries and at-least-once delivery safe.
Unknown outcome	A timeout. The work may or may not have happened, so it must be treated as possibly done.
Eventual consistency	The system is briefly inconsistent (a slot booked for a saga that will fail) but converges to a correct state.
BPMN 2.0	OMG standard notation plus XML format for executable business processes.
Process definition / instance	The deployed diagram (like a class) and one run of it (like an object) with its own tokens and variables.
Token	A marker for "execution is here". Parallel gateways create several; end events consume them.
Wait state	A node where the token parks and the instance is persisted: user task, timer, message, parallel join.
Gateway	Exclusive (XOR) picks one path by condition; parallel (AND) forks into all paths and joins them back.
Boundary event	An event attached to a task: catches an error (⚡), links a compensation handler (⏪), or (in full engines) fires a timer.
BPMN error vs. incident	A modelled business outcome the diagram routes, vs. an unmodelled technical failure that waits for an operator.
Handler / worker	The code behind a service task. The diagram says *what* and *when*; the handler says *how*.
Modeler / engine / cockpit	The three parts of every BPMN tool: draw it, run it, operate it.